IPA Project Report (Phase II)
Implementation of a 64-bit RISC-V CPU

Ritvik Avula
Roll No: 2024112019

Pavani Malchuru
Roll No: 2024102043

Nikhilesh Nallavelli
Roll No: 2024102051

***Abstract*—This report presents the next phase in the design and implementation of a 64-bit RISC-V CPU using Verilog. In this report, we provide a comprehensive overview of the pipelined architecture and detail the different kinds of hazard handling mechanisms that were implemented in the project.**

I. Introduction and Background

THE RISC-V ISA is an open source instruction set architecture that focuses on maintaining simple hardware and using basic instructions in order to design and program more complicated entities. This simplicity makes RISC-V an ideal choice for custom processor design. The instruction set is modular, allowing implementations to support only the necessary base instructions (RV64I) or extend with optional extensions for floating-point operations, atomic instructions, and more. In this project, we focus on implementing the RV64I base integer instruction set for 64-bit systems.

Of all the instructions in the base RV64I ISA, we've implemented the following subset of instructions:

- **R-type** (add, add/or/xor, sll/srl/sra)
- **I-type** (addi and etc..., ld, jalr)
- **S-type** (sd)
- **B-type** (beq)
- **J-type** (jal)

II. Overview of the Pipelined Architecture

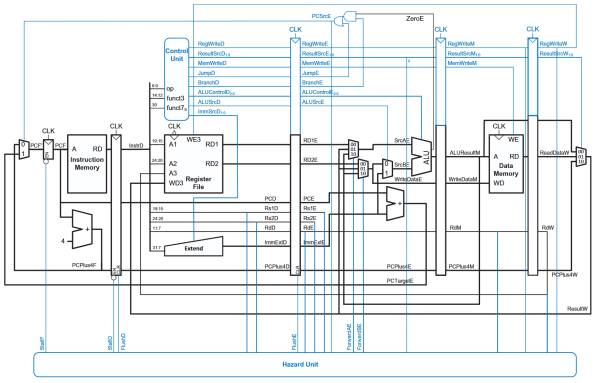

Fig. 1: High-level architecture of the RV64I pipelined processor. Courtesy of *Digital Design and Computer Architecture,* [1]

The RV64I CPU consists of several key modules that work in conjunction to execute instructions. Fig. 1 illustrates the high-level architecture of the processor.

The datapath consists of the following main components:

1) **Program Counter (PC):** Stores the address of the current instruction and increments to fetch the next instruction
2) **Instruction Memory:** Stores the program instructions
3) **Register File:** Contains 32 general-purpose 64-bit registers
4) **Arithmetic Logic Unit (ALU):** Performs computations - either arithmetic or to declare flags to be used in conditional operations
5) **Control Unit:** Decodes and instruction and generates control signals that determine which modules should be active during the instruction cycle
6) **Immediate Generator:** Extracts and extends immediate values from instructions

In this part of the project, we've added a **Hazard Detection Unit** to account for various hazards that pop up when pipelining a processor - namely hazards like:

1) **Data Hazards:** Resolves most RAW data hazards by introducing forwarding paths from MEM and WB stages to the EX stage inputs.
2) **Control Hazards:** Detects branch control hazards and flushes the buffers between stages if needed.

III. Solving Hazards

This section goes into more detail about how different types of hazards are solved in the pipelined CPU.

A. Data Hazards

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed executing. In a pipelined processor, multiple instructions are in different stages simultaneously - which can lead to situations where data is read before it has been written.

In our version of a pipelined CPU, we majorly try to solve **Read-After-Write (RAW) Hazards** - In this type of data hazard, an instruction tries to read a register before a previous instruction writes to it.

*1) **RAW Hazards and Forwarding**:* The textbook *Computer Organization And Design* [2] describes two major pairs of RAW hazards:

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

Mentioned below are two types of RAW hazards that can be solved by forwarding.

```
# 1. mem->ex data hazard
addi x5 x0 5
and x6 x5 x5
# x6 = 5 if works, otherwise x6 = 0
```

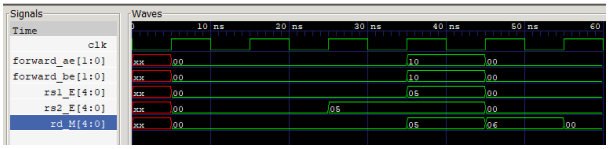

Fig. 2: mem - ex data hazard

Without hazard handling, when `and` reaches the EX stage and attempts to read `x5`, the `addi` instruction may still be in the MEM stage, meaning the updated value of `x5` has not yet been written back to the register file. During the 5th cycle, the `addi` instruction is in the MEM stage, and needs to forward its data to the `and` instruction in the **EX stage**. Notice in fig. 2, both the the forward signals get asserted when rs1_E = rd_M = rs2_E.

Similarly, considering the following code snippet:

```
# 2. wb->ex data hazard
addi x5 x0 5
addi x0 x0 0
and x6 x5 x5
# x6 = 5 if works, otherwise x6 = 0
```

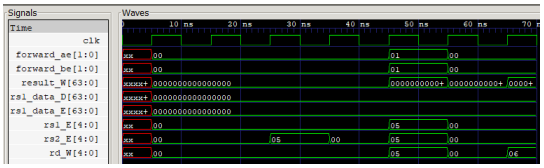

Fig. 3: WB-EX Forwarding

Again, here, when `and` reaches the EX stage on cycle 5, `x5` is still in the WB stage - when it has to be reached to EX for the `and` to execute correctly.

We can fix these problems by **forwarding the data**. The forwarding unit compares the source registers of the instruction in the EX stage (`rs1_E`, `rs2_E`) against destination registers in later stages (`rd_M` in MEM stage and `rd_W` in WB stage). When a match is detected and the corresponding RegWrite signal is enabled, the forwarding unit selects the computed value directly from the later stage instead of using the stale value from the register file.

Two forwarding select signals are generated:

- **ForwardA:** Controls which value feeds ALU input A.
  - `00`: Use register file output (no hazard)
  - `01`: Forward from WB
  - `10`: Forward from MEM
- **ForwardB:** Controls which value feeds ALU input B. (same encoding)

Note that when comparing register addresses, both forwarding paths may be active simultaneously. Consider the following assembly code as an example.

```
# Prioritzing mem->ex over wb->ex test
addi x5 x0 5
addi x5 x5 1 # x5 = 6
and x6 x5 x5
```

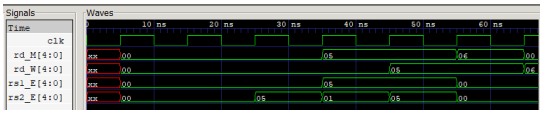

Fig. 4: Priority during simultaneous forwarding cases

In the example from above, it is important that the **more recent result be given priority.** If MEM-EX forwarding was not given priority, `x6`'s value would've been 5 or 0 - not 6 like it is actually supposed to be by the end of line 2. fig 4 shows that in the 5th cycle both `forward` signals were asserted to `10` rather than `01`.

The code snippet that implements the forwarding looks like this:

```
if(((rs1_e == rd_m) && reg_write_m) &&
   (rs1_e != 0)
) // EX-MEM hazard
      forward_ae = 2'b10;
  else if(((rs1_e == rd_w) && reg_write_w)
  && (rs1_e != 0)) // MEM-WB hazard
      forward_ae = 2'b01;
  else
      forward_ae = 2'b00;
```

Of course, the same follows for `rs2` and `forward_be`.

Bonus: About Load-to-Store Forwarding

An important special case occurs when a load instruction is immediately followed by a store instruction that uses the loaded value:

```
# wb->mem forwarding test
addi x5 x0 8
sd x5 0(x0) # mem[0] = 8
ld x6 0(x0)
# x6 = 8 if works, otherwise x6 = 0
```

In this scenario, the store instruction in the MEM stage needs the value from register `x5` as its write data. Since the load result becomes available at the end of the MEM stage (or from the MEM/WB pipeline register), we implement a forwarding path that routes the loaded data directly to the store's write data input.
The following code snippet shows how we fixed this issue.

```
// ld -> sd forwarding
if(((rd_w == rs2_m) && reg_write_w &&
mem_write_m) && (rs2_m != 0))
    forward_m = 1'b1;
```

This eliminates the need for the data to first pass through the Writeback stage and register file, allowing back-to-back load-store operations without stalling.

This forwarding mechanism eliminates most RAW hazards without stalling the pipeline.

*2) **Load-Use Hazards and Stalling**:* Forwarding cannot solve all RAW hazards. A critical exception is the **load-use hazard**, where a load instruction is immediately followed by an instruction that uses the loaded data:

```
ld x5, 0(x1)       # Load x5 from memory
add x6, x5, x2     # Immediately use x5
```

The problem is that the data from memory becomes available only at the end of the MEM stage, but the `add` instruction needs it at the beginning of the EX stage—one cycle too early. Forwarding paths exist, but the data simply isn't ready yet.

To fix this, we introduced stall and flush signals to temporarily disable/clear the registers - this lets us introduce "bubbles" into the pipeline that let the load instruction execute in MEM before the next instruction reaches EX - then the MEM-EX forwarding path can be utilized. Hence:

```
always @ (*) begin
    stall = 1'b0;
    flush = 1'b0;
    if ((mem_to_reg_e == 2'b01) &&
    (mem_write_d == 1'b0) && (rd_e != 0) &&
    ((rs1_d == rd_e) || (rs2_d == rd_e)))
    begin
        stall = 1'b1;
    end
    // Control hazard detection (for beq and
    jal)
    else if (PCSrc_E) begin //pc_src_mem =
    (zero_flag & branch_signal) | jump
        flush = 1'b1;
    end
end
```

Note that we check if `mem_write_d == 1'b0`, so that we don't accidentally stall the pipeline for a load-store data hazard as we discussed earlier.

This single-cycle penalty is unavoidable for load-use hazards but minimizes performance impact compared to more conservative stalling strategies.

B. Control Hazards

Control hazards (also called branch hazards) occur when the pipeline must make decisions about instruction flow before determining whether a branch will be taken. In a pipelined processor, by the time a branch instruction reaches the stage where its condition is evaluated, several subsequent instructions have already been fetched into the pipeline. If the branch is taken, these speculatively fetched instructions are incorrect and must be discarded.

Consider this example:

```
    beq x1, x2, target     # Branch if x1 == x2
    add x3, x4, x5         # Next sequential
        instruction
    sub x6, x7, x8         # Another sequential
         instruction
target:
    or x9, x10, x11        # Branch target
```

1) Static Branch Prediction: Static branch prediction is a simple technique that predicts the outcome of a branch instruction without using any type of runtime history. It is called static because the prediction is always the same irrespective of the assembly code. After execution of the branch instruction, if the prediction was correct, the pipeline continues without interruption. However, if the prediction was incorrect (i.e., the branch is taken when we predicted not taken), the pipeline must flush all instructions that were fetched after the branch and fetch the correct instructions from the branch target address. This flushing process incurs a performance penalty, as it wastes cycles on instructions that will not be executed.

IV. Implementation Details

A. Structure

Each module of the CPU has its own designated folder, containing:

- The file containing the module itself (`<module>.v`)
- A testbench to validate each of the functions of that module (`<module>_tb.v`)
- A bunch of intermediary files (`.vvp, .vcd`) that plot outputs

Apart from these folders, we also have a folder for testcases, written in assembly(`.asm files`) - however, `.txt` files also work.

1) Assembler and its Features: We've also written a custom assembler in C - to translate RISC-V assembly code into machine code. It reads assembly instructions from `.asm` or `.txt` files, parses each instruction, and generates the corresponding 32-bit machine code that can be executed by the CPU.

The Assembler supports the following key features:

- **Instruction Parsing:** Recognizes all R-type, I-type, S-type, B-type, and J-type instructions implemented in the CPU
- **Register Mapping:** Only the general naming convention (x0, x5, etc.) works.
- **Immediate Encoding:** Encodes immediate values and performs sign extension where necessary
- **Machine Code Generation:** Outputs 32-bit instruction encodings in a format compatible with the instruction memory module

The assembler operates in two passes:

1) **First Pass:** Scans the assembly file to identify and record the addresses of all labels
2) **Second Pass:** Translates each instruction to machine code, using the label table to resolve branch and jump offsets

The output is written to a text file where each line represents one byte of the instruction in hexadecimal format.

And finally, the folder also contains a script that assembles the .asm files, runs them on the CPU and generates all the required output files.

This project successfully demonstrates the design and implementation of a 64-bit RISC-V processor core. The modular architecture allows for clear separation of concerns and facilitates future extensions. All modules have been thoroughly

tested and verified to operate correctly. The Phase 1 design provides a solid foundation for implementing a pipelined processor in subsequent phases.

REFERENCES

[1] S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.
[2] D. A. Patterson and J. L. Hennessy, *Computer organization and Design*. Morgan Kaufmann, 2022.
[3] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
[4] DownToTheWires, "Risc-v intro and r-type alu instructions," 2025, accessed: 2025. [Online]. Available: https://www.youtube.com/@DowntotheWires